ASW 2025/2026 · CHECKPOINT 1

RunX

Plataforma Web de Gestão Diária de Corridas

DOCUMENTO	GRUPO	ENTREGA
Especificação 1ª Fase	4 elementos	18 de Março de 2026

Índice

01 · ENQUADRAMENTO

Enquadramento

1.1 Apresentação do Projeto

O **RunX** é uma plataforma Web de gestão diária de corridas que combina o registo individual de treinos com dinâmicas sociais e colaborativas. A aplicação permite que os utilizadores registem as suas corridas, acompanhem o seu progresso semanal e partilhem experiências com outros corredores — individualmente ou em grupo.

A plataforma integra informação contextual (localização, tempo, clima, rotinas) com funcionalidades de arquivo vivo, permitindo que cada corrida se torne não apenas um registo de desempenho, mas também uma memória narrativa associada a um lugar, um estado de espírito ou um tema.

1.2 Problema a Resolver

As aplicações de corrida existentes focam-se maioritariamente em métricas de desempenho (distância, ritmo, calorias), ignorando a dimensão narrativa e social do ato de correr. Por outro lado, as plataformas sociais de fitness carecem de ferramentas de organização pessoal e contextualização temporal.

O RunX resolve esta lacuna ao:

- Combinar registo de desempenho com memória narrativa (foto, observações, frase motivacional)
- Integrar lembretes contextuais com base em rotinas, localização e progresso semanal
- Permitir corridas coletivas com partilha de trajetos e publicações conjuntas
- Sugerir trajetos e analisar padrões usando inteligência artificial

1.3 Tópicos Escolhidos e Motivação

> **Mix de dois tópicos do enunciado**
>
> **Gestão de Tarefas Contextual e Social** — o ato de correr é tratado como uma tarefa recorrente, com contexto (tempo, localização, rotina, estado) e dimensão social (grupos, partilha, corridas coletivas, sugestões entre utilizadores).
>
> **Life Threads: Diário Digital Colaborativo** — cada corrida torna-se um registo no arquivo vivo do utilizador, com foto, observações, tema e frase motivacional, pesquisável e partilhável, construindo uma narrativa evolutiva de experiências pessoais e coletivas ao longo do tempo.

Esta combinação é natural: correr é simultaneamente um hábito a gerir e uma experiência a recordar. O RunX explora essa dualidade para criar uma plataforma coerente que vai além do simples tracking de atividade física.

02 · UTILIZADORES

Utilizadores e Cenários

2.1 Tipos de Utilizadores

O sistema define dois perfis principais com permissões distintas:

Perfil	Corredor (Runner)	Organizador de Grupo
DESCRIÇÃO	Utilizador padrão da plataforma. Regista corridas, acompanha o progresso semanal, partilha publicações e participa em eventos coletivos.	Utilizador com permissões elevadas para gerir grupos. Cria eventos coletivos, modera publicações do grupo e convida membros.
PERMISSÕES	CRUD sobre as suas próprias corridas; leitura do feed social; participação em grupos; gestão do seu perfil.	Todas as permissões do Corredor, mais: criar/editar/eliminar grupos; criar eventos coletivos; gerir membros; moderar conteúdo do grupo.
ACESSO API	`/runs`, `/profile`, `/feed`, `/groups` (leitura e participação)	Endpoints adicionais: `/groups` (escrita), `/groups/:id/events`, `/groups/:id/members`

2.2 Cenários de Utilização Principais

Cenário 1 — Registo diário de corrida (Corredor)

A Sofia é corredora habitual e usa o RunX para registar os seus treinos diários. Na sexta-feira ao fim da tarde, recebe uma notificação push do RunX a lembrar-lhe que faltam 5.8 km para atingir a meta semanal de 40 km. A Sofia sai para correr pela Ribeira de Lisboa, com o GPS ativo na app.

No final da corrida, abre o RunX e vê as métricas registadas automaticamente (6.2 km, 52 min, ritmo de 8:23/km). Tira uma fotografia ao pôr do sol sobre o Tejo, seleciona o tema *Amanhecer*, escreve uma frase motivacional e partilha a publicação com o seu grupo *Ribeira Runners*. O dashboard atualiza-se imediatamente, mostrando a meta semanal atingida.

Cenário 2 — Organização de corrida coletiva (Organizador)

O Pedro é organizador do grupo *Monsanto Trail Crew*. Decide criar um evento coletivo para o sábado seguinte: uma corrida de 10 km no Parque de Monsanto com o tema *Natureza*. Na página de gestão do grupo, cria o evento, define o trajeto no mapa e envia convites aos 12 membros do grupo.

Os membros recebem uma notificação push com o convite. Oito confirmam presença. No dia do evento, cada corredor regista a sua corrida individualmente, mas todas ficam associadas ao evento coletivo, gerando uma publicação conjunta no feed.

Cenário 3 — Exploração do histórico e sugestão de trajeto (Corredor)

O João quer explorar o seu arquivo de corridas. Na secção Histórico, pesquisa por *'Monsanto'* e filtra pelo tema *Natureza*. A app mostra-lhe todas as corridas naquele parque, com os mapas dos trajetos e as notas que foi escrevendo. Ao abrir uma corrida, a funcionalidade de LLM sugere-lhe uma variante do trajeto com 2 km adicionais, baseada nos seus padrões de corrida e preferências do perfil.

Cenário 4 — Análise semanal e relatório com LLM (Corredor)

No final de cada semana, o RunX gera automaticamente um relatório personalizado com análise dos padrões de corrida: dias mais produtivos, ritmo médio por tema, evolução mensal e uma sugestão de meta para a semana seguinte. O relatório inclui também uma frase motivacional gerada por LLM com base nas corridas registadas durante a semana.

03 · STORYBOARDS

Storyboards e Sketches de Interação

Os storyboards foram desenvolvidos como documento HTML interativo, cobrindo **12 ecrãs** organizados em **6 fluxos principais**. O ficheiro `storyboard-corridas.html` encontra-se em anexo digital.

Fluxo	Nome	Ecrãs incluídos
Fluxo 01	**Entrada & Dashboard Semanal**	Login, Dashboard com strip semanal, Notificações push contextuais
Fluxo 02	**Registar & Documentar uma Corrida**	Dados GPS + mapa, Foto + memória (tema, frase), Publicação social, Confirmação
Fluxo 03	**Histórico, Pesquisa & Exploração**	Timeline de corridas com filtros por tema, Detalhe completo de corrida
Fluxo 04	**Feed Social & Corridas Coletivas**	Feed agregado com publicações, Evento coletivo (criação e participação)
Fluxo 05	**Diagrama de Navegação**	Visão geral das transições entre todos os ecrãs
Fluxo 06	**Perfil do Utilizador**	Vista do perfil com estatísticas, Editar perfil (avatar, username, password), Definições e notificações

Nota sobre os storyboards

O documento HTML dos storyboards encontra-se em anexo digital (`storyboard-corridas.html`). Contém sketches interativos de alta fidelidade para todos os ecrãs descritos acima, com anotações detalhadas por ecrã e diagrama de navegação completo.

04 · CASOS DE USO

Casos de Uso

Os casos de uso descrevem as interações principais entre os utilizadores e o sistema, com pré-condições e pós-condições explícitas conforme o modelo UP (Unified Process).

UC-01 Registar Corrida	Corredor
PRÉ-CONDIÇÕES	Utilizador autenticado; GPS ativo no dispositivo.
PÓS-CONDIÇÕES	Corrida guardada na BD com métricas, trajeto, foto e tema; progresso semanal atualizado; publicação criada (se visibilidade ≠ privado).
FLUXO PRINCIPAL	1. Utilizador inicia o registo de corrida na app. 2. Sistema começa a recolher dados GPS (distância, ritmo, tempo). 3. Utilizador termina a corrida. 4. Sistema calcula métricas finais e apresenta o formulário de memória. 5. Utilizador faz upload de foto, escreve observações e seleciona tema. 6. LLM sugere frase motivacional (pré-preenchida, editável). 7. Utilizador define visibilidade e confirma publicação. 8. Sistema guarda a corrida, atualiza o dashboard semanal e cria o post.
ALTERNATIVO	Falha de GPS: sistema permite inserção manual de distância e tempo.

UC-02	Consultar Histórico de Corridas	Corredor

PRÉ-CONDIÇÕES	Utilizador autenticado; pelo menos uma corrida registada.
PÓS-CONDIÇÕES	Lista de corridas apresentada com filtros aplicados; detalhe de corrida acessível.
FLUXO PRINCIPAL	1. Utilizador navega para a secção Histórico. 2. Sistema apresenta timeline de corridas ordenada por data. 3. Utilizador aplica filtros (tema, localização, intervalo de datas, texto livre). 4. Sistema filtra e apresenta os resultados correspondentes. 5. Utilizador seleciona uma corrida para ver o detalhe. 6. Sistema apresenta métricas, mapa do trajeto, foto e observações.
ALTERNATIVO	Sem resultados para os filtros aplicados: sistema apresenta mensagem informativa.

UC-03	Criar Evento Coletivo	Organizador de Grupo

PRÉ-CONDIÇÕES	Utilizador autenticado com `role=organizer`; grupo existente com pelo menos 2 membros.
PÓS-CONDIÇÕES	Evento criado e guardado na BD; notificação push enviada a todos os membros do grupo.
FLUXO PRINCIPAL	1. Organizador acede à página de gestão do grupo. 2. Organizador clica em 'Criar Evento' e preenche: título, data/hora, trajeto no mapa, tema. 3. Sistema valida os dados e cria o evento. 4. Sistema envia notificação push (via FCM) a todos os membros. 5. Membros podem confirmar ou recusar presença.
ALTERNATIVO	Erro no envio de notificações: evento é criado, notificações são colocadas em fila para reenvio.

UC-04	Gerir Membros do Grupo	Organizador de Grupo
PRÉ-CONDIÇÕES	Utilizador autenticado com `role=organizer` no grupo em questão.	
PÓS-CONDIÇÕES	Membro adicionado ou removido do grupo; lista de membros atualizada.	
FLUXO PRINCIPAL	1. Organizador acede à página de gestão de membros do grupo. 2. Para adicionar: pesquisa utilizador por username e envia convite. 3. Utilizador convidado recebe notificação e aceita/recusa. 4. Para remover: organizador seleciona membro e confirma remoção. 5. Sistema atualiza a tabela `group_members`.	
ALTERNATIVO	Utilizador a remover é o único organizador: sistema bloqueia a ação e apresenta aviso.	

UC-05	Obter Sugestão de Trajeto via LLM	Corredor
PRÉ-CONDIÇÕES	Utilizador autenticado; corrida com trajeto GPS registado.	
PÓS-CONDIÇÕES	Sugestão de variante de trajeto gerada e apresentada; guardada na BD associada à corrida.	
FLUXO PRINCIPAL	1. Utilizador abre o detalhe de uma corrida e clica em 'Sugerir variante'. 2. Sistema envia ao LLM: trajeto atual, histórico de corridas na área, preferências do utilizador. 3. LLM devolve descrição textual da variante com pontos de interesse e distância estimada. 4. Sistema guarda a sugestão e apresenta-a ao utilizador no mapa.	
ALTERNATIVO	Falha da API LLM: sistema apresenta mensagem de indisponibilidade temporária.	

UC-06 Editar Perfil e Configurações	Corredor / Organizador
PRÉ-CONDIÇÕES	Utilizador autenticado.
PÓS-CONDIÇÕES	Dados do perfil atualizados na BD; novo avatar carregado para o serviço de armazenamento (se alterado).
FLUXO PRINCIPAL	1. Utilizador acede à página de perfil e clica em 'Editar'. 2. Sistema apresenta formulário com dados atuais. 3. Utilizador altera campos desejados (username, bio, avatar, localização). 4. Para alterar password: utilizador introduz password atual e nova password (×2). 5. Sistema valida e guarda as alterações. 6. Se avatar alterado: imagem enviada para Cloudinary/S3, URL atualizado na BD.
ALTERNATIVO	Username já existente: sistema apresenta erro de validação sem guardar.

05 · SISTEMA

Sistema e Funcionalidades

5.1 Recursos do Sistema

Recurso	Descrição
User	Conta de utilizador com perfil, preferências e permissões (runner / organizer)
Run	Registo de uma corrida: métricas GPS, trajeto, foto, observações, tema, frase motivacional
Group	Grupo de corredores com membros e organizador(es)
GroupEvent	Evento coletivo criado por um organizador, associado a um grupo
Post	Publicação social derivada de um Run ou GroupEvent, visível no feed
Theme	Etiqueta temática associada a uma corrida (ex: Natureza, Urbano, Amanhecer)
Notification	Lembrete ou alerta gerado pelo sistema ou por atividade social
WeeklyGoal	Meta semanal de km definida pelo utilizador, com progresso calculado
RouteVariant	Sugestão de variante de trajeto gerada por LLM
WeeklyReport	Relatório semanal gerado por LLM com análise de padrões

5.2 Funcionalidades Principais

Gestão de Corridas

- Registo de corrida com métricas automáticas (distância, tempo, ritmo, calorias) via GPS
- Upload de fotografia da corrida

- Adição de observações livres, frase motivacional e tema
- Consulta e pesquisa do histórico de corridas (por data, localização, tema, texto livre)
- Visualização de trajeto GPS em mapa interativo

Progresso e Metas

- Dashboard semanal com strip de dias, barra de progresso e meta configurável
- Gráfico de km mensais no perfil
- Conquistas (badges) desbloqueadas por milestones
- Sequência de dias consecutivos com corrida registada (streak)

Social e Colaboração

- Feed social com publicações de corridas da rede do utilizador
- Reações e comentários em publicações
- Grupos de corrida com gestão de membros pelo organizador
- Criação de eventos coletivos com trajeto, data/hora e tema partilhado
- Publicação conjunta automática após evento coletivo
- Reutilização de trajetos de outros utilizadores ("Usar trajeto")

Contexto e Notificações

- Lembretes diários configuráveis por hora e dias da semana
- Notificações push contextuais (progresso, atividade social, sugestões de trajeto)
- Registo automático de condições meteorológicas no momento da corrida (OpenWeather)

Perfil e Conta

- Edição de avatar, username, nome, bio e localização
- Alteração de password e email
- Estatísticas totais: km, horas, nº de corridas, data de criação da conta
- Configuração de unidades (km/milhas) e visibilidade do perfil

Funcionalidades com LLM

- Sugestão automática de variantes de trajeto baseada em histórico e preferências
- Geração de frases motivacionais personalizadas no registo de corrida
- Análise de padrões e geração de relatório semanal personalizado

5.3 Fluxos Principais do Sistema

Fluxo	Sequência
Registo de corrida	Autenticação → Dashboard → Nova Corrida (GPS) → Foto + Memória → Publicar/Guardar → Dashboard atualizado
Descoberta e exploração	Dashboard → Histórico → Pesquisa/Filtro → Detalhe de Corrida → Sugestão de variante (LLM)
Corrida coletiva	Organizador cria evento → Membros recebem convite → Confirmação de presença → Corrida individual → Publicação conjunta gerada
Relatório semanal	Fim de semana → LLM analisa corridas → Gera relatório + frase motivacional → Notificação push enviada
Gestão de grupo	Organizador cria grupo → Convida membros → Define evento → Gere membros e modera conteúdo

06 · REQUISITOS

Requisitos Funcionais e Não Funcionais

6.1 Requisitos Gerais do Sistema

ID	Requisito	Fase	Tipo
RG-01	Plataforma multiutilizador com autenticação por JWT	F1	Plataforma
RG-02	Dois tipos de utilizador: Corredor e Organizador de Grupo, com permissões distintas	F1	Plataforma
RG-03	Fluxos com interação entre utilizadores (feed, grupos, eventos)	F1	Plataforma
RG-04	Backend implementado em Node.js com Express	F2	Backend
RG-05	Persistência de dados com PostgreSQL	F2	Backend
RG-06	Modelo de dados coerente com os recursos definidos	F2	Backend
RG-07	Operações CRUD sobre todos os recursos principais	F2	Backend
RG-08	API organizada em camadas: routing, lógica de negócio, acesso a dados	F2	API
RG-09	Documentação da API com OpenAPI/Swagger	F2	API
RG-10	Frontend implementado em React, integrado com a API	F3	Frontend
RG-11	Integração com pelo menos um serviço externo	F3	Integrações
RG-	Integração com funcionalidade baseada em LLM	F3	Integrações

12			
RG-13	Autenticação e controlo de acesso a todos os recursos protegidos	F2	Segurança
RG-14	Validação de dados de entrada em todos os endpoints	F2	Segurança
RG-15	Tratamento consistente de erros com respostas HTTP adequadas	F2	Segurança
RG-16	Configuração via variáveis de ambiente (.env)	F2	Segurança

6.2 Requisitos por Fase

F1 — Especificação (18 Mar)

ID	Requisito	Tipo
RF-01	Documento de especificação com enquadramento, utilizadores e cenários	Funcional
RF-02	Storyboards e sketches de interação para todos os fluxos principais	Funcional
RF-03	Lista de funcionalidades, recursos e requisitos definidos	Funcional
RF-04	Modelo de dados preliminar e exemplos de endpoints da API	Técnico
RF-05	Identificação de serviços externos e ideias para LLM	Técnico

F2 — Backend e Serviços

ID	Requisito	Tipo
RF-06	API REST completa com autenticação JWT e controlo de acesso	Funcional
RF-07	Endpoints CRUD para: Users, Runs, Groups, GroupEvents, Posts	Funcional
RF-08	Persistência em PostgreSQL com migrações e modelo relacional	Técnico
RF-09	Validação de inputs e tratamento de erros em todos os endpoints	Técnico
RF-10	Documentação OpenAPI/Swagger disponível em `/api-docs`	Técnico
RF-11	Integração inicial com OpenWeather API (clima na corrida)	Integração

F3 — Integração e Frontend

ID	Requisito	Tipo
RF-12	Frontend React com todos os fluxos principais implementados	Funcional
RF-13	Dashboard semanal, registo de corrida, histórico e feed social	Funcional
RF-14	Integração com Mapbox/Google Maps para mapas e trajetos GPS	Integração
RF-15	Integração com Cloudinary/S3 para upload de imagens	Integração
RF-16	Integração com Firebase Cloud Messaging para notificações push	Integração
RF-17	Integração com LLM para sugestão de trajetos e frases motivacionais	LLM

F4 — Versão Final

ID	Requisito	Tipo
RF-18	Relatório semanal personalizado gerado por LLM	LLM
RF-19	Sistema de conquistas (badges) e sequências de dias	Funcional
RF-20	Testes de integração nos fluxos principais	Técnico
RF-21	Documentação final e discussão de decisões arquiteturais	Técnico

6.3 Requisitos Não Funcionais

ID	Categoria	Requisito
RNF-01	Desempenho	A API deve responder a pedidos simples em menos de 300ms em condições normais de carga.
RNF-02	Desempenho	O sistema deve suportar pelo menos 50 utilizadores simultâneos sem degradação visível.
RNF-03	Segurança	Todas as passwords são armazenadas com hash bcrypt (fator de custo ≥ 10). Nunca em texto simples.
RNF-04	Segurança	Tokens JWT expiram ao fim de 1 hora; refresh tokens ao fim de 7 dias. Comunicação apenas por HTTPS.
RNF-05	Segurança	Todas as operações de escrita validam que o utilizador autenticado tem permissão sobre o recurso (controlo de acesso por ownership).
RNF-06	Disponibilidade	O sistema deve ter disponibilidade de 99% durante o período de avaliação do projeto.
RNF-07	Usabilidade	A interface deve ser responsiva e funcional em browsers modernos (Chrome, Firefox, Safari) em desktop e mobile.
RNF-08	Manutenibilidade	O código deve seguir uma estrutura em camadas clara (routes / controllers / services / models) e estar documentado nos pontos críticos.
RNF-09	Manutenibilidade	Toda a configuração sensível (chaves de API, strings de ligação à BD, segredos JWT) deve ser gerida por variáveis de ambiente (.env), nunca em código fonte.
RNF-10	Portabilidade	A aplicação deve poder ser executada localmente com um único comando (ex: `docker-compose up`) ou com instruções claras no README.

RNF-11	Resiliência	Falhas em serviços externos (LLM, OpenWeather, FCM) não devem bloquear o fluxo principal. O sistema degrada graciosamente com mensagens informativas ao utilizador.
RNF-12	Dados (RGPD)	O sistema deve suportar eliminação de conta com remoção de dados pessoais. Publicações em grupos ficam anonimizadas, não são eliminadas.

07 · ARQUITETURA

Arquitetura Inicial e Modelo E-A

7.1 Visão Geral da Arquitetura

O RunX segue uma arquitetura cliente-servidor em três camadas, com uma API REST no centro do sistema:

- **Frontend (React)** — SPA servida ao cliente, consome a API via HTTP
- **Backend (Node.js + Express)** — API REST organizada em rotas, controladores e serviços
- **Persistência (PostgreSQL)** — base de dados relacional com modelo normalizado
- **Serviços Externos** — integrados no backend, não expostos diretamente ao frontend

7.2 Organização do Backend

Pasta	Responsabilidade
`routes/`	Definição dos endpoints e middlewares de autenticação/autorização
`controllers/`	Lógica de tratamento de pedidos e respostas HTTP
`services/`	Lógica de negócio: validações, regras, cálculos
`models/`	Queries à base de dados (ou ORM) e definições de tipos
`integrations/`	Clientes para serviços externos (Maps, Weather, LLM, Storage, FCM)
`middlewares/`	Autenticação JWT, controlo de acesso, validação, erros
`config/`	Variáveis de ambiente, configuração de base de dados e serviços

7.3 Diagrama Entidade-Associação

Diagrama E-A preliminar do RunX com 13 entidades, atributos principais e relações com

cardinalidades. As entidades estão organizadas em 4 colunas temáticas: Corridas · Grupos · Social · Auxiliares.

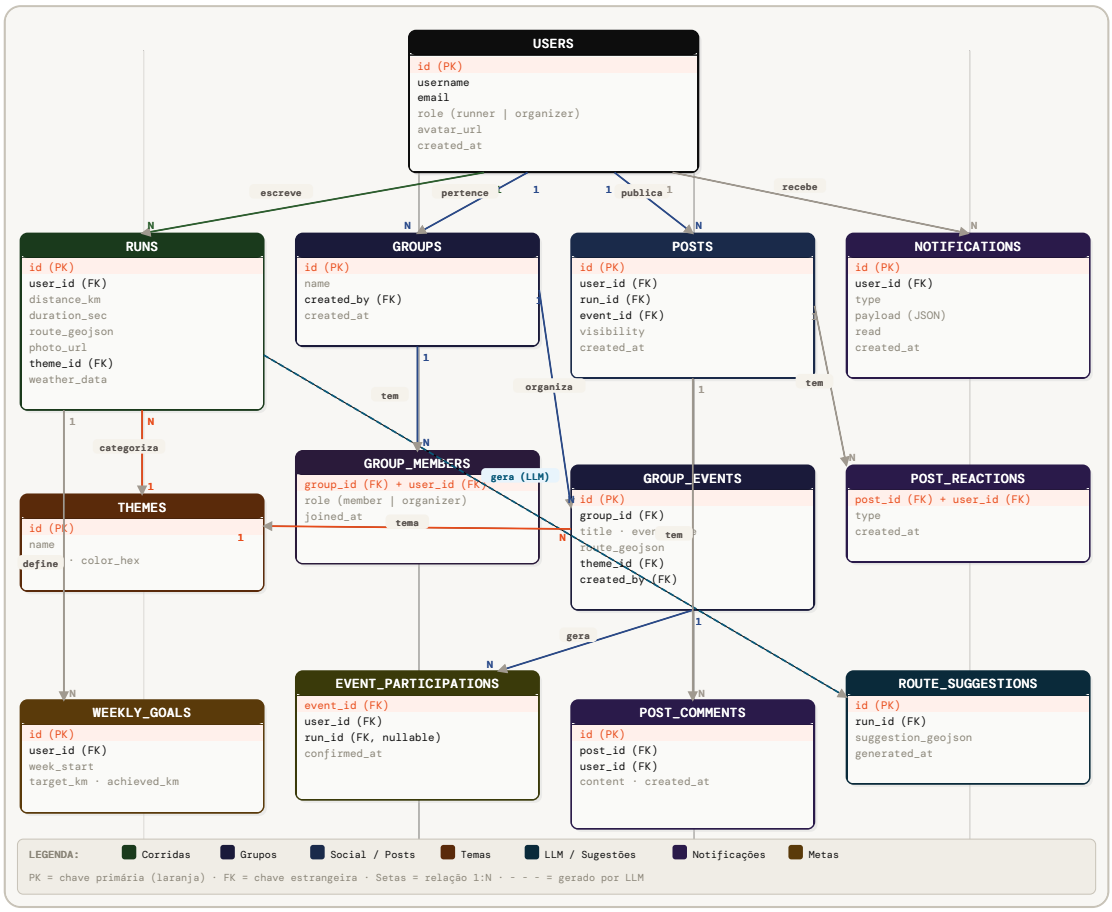

Diagrama E-A preliminar do RunX — 13 entidades, relações com cardinalidades. Um diagrama formal em notação UML/Chen será produzido na Fase 2.

7.4 Exemplos de Endpoints da API

Autenticação

Método	Endpoint	Descrição	Auth
POST	`/auth/register`	Registar novo utilizador	—
POST	`/auth/login`	Autenticar e obter JWT	—
POST	`/auth/logout`	Invalidar token	Auth
POST	`/auth/refresh`	Renovar JWT	Auth

Utilizadores e Perfil

Método	Endpoint	Descrição	Auth
GET	/users/me	Perfil do utilizador autenticado	Auth
PUT	/users/me	Atualizar perfil (nome, bio, localização)	Auth
PUT	/users/me/password	Alterar password	Auth
POST	/users/me/avatar	Upload de avatar (multipart)	Auth
GET	/users/:id	Perfil público de outro utilizador	Auth
GET	/users/me/stats	Estatísticas totais do utilizador	Auth
DELETE	/users/me	Eliminar conta (anonimiza posts em grupos)	Auth

Corridas (Runs)

Método	Endpoint	Descrição	Auth
GET	/runs	Listar corridas do utilizador (com filtros)	Auth
POST	/runs	Criar nova corrida	Auth
GET	/runs/:id	Detalhes de uma corrida	Auth
PUT	/runs/:id	Editar corrida (observações, frase, tema)	Auth
DELETE	/runs/:id	Eliminar corrida	Auth
GET	/runs/week	Corridas e progresso da semana atual	Auth
GET	/runs/:id/suggest-route	Sugerir variante de trajeto via LLM	Auth

Grupos e Eventos Coletivos

Método	Endpoint	Descrição	Auth
GET	/groups	Listar grupos do utilizador	Auth
POST	/groups	Criar novo grupo	Auth + Org
GET	/groups/:id	Detalhes de um grupo	Auth
POST	/groups/:id/members	Adicionar membro ao grupo	Auth + Org
DELETE	/groups/:id/members/:uid	Remover membro	Auth + Org
GET	/groups/:id/events	Listar eventos do grupo	Auth
POST	/groups/:id/events	Criar evento coletivo	Auth + Org
POST	/groups/:id/events/:eid/join	Confirmar presença no evento	Auth

Feed Social

Método	Endpoint	Descrição	Auth
GET	/feed	Feed social do utilizador	Auth
POST	/posts/:id/react	Reagir a uma publicação	Auth
POST	/posts/:id/comments	Comentar numa publicação	Auth

Funcionalidades LLM

Método	Endpoint	Descrição	Auth
POST	/llm/motivational-phrase	Gerar frase motivacional personalizada	Auth
GET	/users/me/weekly-report	Obter relatório semanal gerado por LLM	Auth
POST	/runs/:id/suggest-route	Sugestão de variante de trajeto por LLM	Auth

08 · FLUXOS DE INTEGRAÇÃO

Fluxos de Integração com Serviços Externos

Os fluxos seguintes descrevem as sequências de ações internas e externas para as funcionalidades que dependem de serviços externos, ilustrando a complexidade real de integração.

8.1 Fluxo: Registar Corrida com Dados Meteorológicos

Envolve o cliente, o backend RunX, a API OpenWeather e opcionalmente o LLM e o FCM. As linhas a laranja identificam chamadas a serviços externos.

#	Interveniente	Ação / Chamada	Detalhe
1	**Cliente → Backend**	`POST /runs`	Envia métricas GPS, coordenadas e timestamp de fim de corrida
2	**Backend**	Validação	Valida campos obrigatórios, verifica JWT e ownership
3	**Backend → OpenWeather**	`GET /weather?lat=&lon=`	Pedido de condições meteorológicas para as coordenadas e hora da corrida
4	**OpenWeather → Backend**	`200 OK {temp, rain, wind}`	Dados meteorológicos recebidos e mapeados para o modelo interno
5	**Backend → LLM** *(opcional)*	`POST /v1/messages`	Se utilizador pediu frase motivacional: envia contexto da corrida
6	**LLM → Backend**	`200 OK {phrase}`	Frase motivacional gerada
7	**Backend → PostgreSQL**	`INSERT INTO runs ...`	Corrida guardada com `weather_data` (JSON) e `motivational_phrase`
8	**Backend → FCM** *(se post público)*	`POST /fcm/send`	Notificação enviada a seguidores sobre nova publicação
9	**Backend → Cliente**	`201 Created {run}`	Resposta com corrida criada, incluindo dados meteorológicos

> Degradação graciosa: se OpenWeather falhar (timeout ou erro), a corrida é guardada sem `weather_data`. O utilizador é informado na resposta.

8.2 Fluxo: Sugestão de Variante de Trajeto via LLM

#	Interveniente	Ação / Chamada	Detalhe
1	**Cliente → Backend**	`GET /runs/:id/suggest-route`	Utilizador clica em 'Sugerir variante' no detalhe da corrida
2	**Backend → PostgreSQL**	`SELECT runs + history`	Recolhe a corrida atual e as últimas 10 corridas do utilizador na mesma área
3	**Backend → PostgreSQL**	`SELECT user preferences`	Recolhe preferências: tema favorito, distância habitual, nível de dificuldade
4	**Backend → LLM API**	`POST /v1/messages`	Envia prompt com trajeto GeoJSON, histórico resumido, preferências
5	**LLM API → Backend**	`200 OK {JSON}`	Devolve JSON estruturado: descrição, pontos de interesse, distância estimada
6	**Backend**	Validação da resposta	Verifica estrutura do JSON; se inválido, retorna 502 com mensagem amigável
7	**Backend → PostgreSQL**	`INSERT INTO route_suggestions`	Guarda sugestão associada ao `run_id`
8	**Backend → Cliente**	`200 OK {suggestion}`	Retorna sugestão para apresentar no mapa e em texto

> Rate limit: máximo 5 sugestões/utilizador/dia para controlo de custos de tokens.

09 · SERVIÇOS E LLM

Serviços Externos e LLM

9.1 Serviços Externos

Google Maps / Mapbox — Mapeamento e GPS

Visualização de trajetos GPS no mapa; cálculo de distâncias; geocodificação de localização; sugestão de percursos base. **Integração prevista na Fase 3.**

Cloudinary / AWS S3 — Armazenamento de Imagens

Upload, redimensionamento e entrega otimizada das fotografias de corrida e avatares de perfil. **Integração prevista na Fase 3.**

Firebase Cloud Messaging — Notificações Push

Envio de lembretes diários configuráveis, alertas de atividade social e convites para eventos coletivos. **Integração prevista na Fase 3.**

OpenWeather API — Dados Meteorológicos

Registo automático das condições climatéricas (temperatura, precipitação, vento) no momento e localização de cada corrida. **Integração prevista na Fase 2.**

9.2 Funcionalidades com LLM

O RunX integra um modelo de linguagem (via API da Anthropic ou OpenAI) para três funcionalidades distintas, todas com prompts específicos e respostas estruturadas:

Sugestão Automática de Trajetos

- **Trigger:** utilizador abre o detalhe de uma corrida e clica em 'Sugerir variante'
- **Input ao LLM:** histórico de corridas na mesma área, distância habitual, tema preferido, nível de dificuldade
- **Output esperado:** descrição textual de uma variante do trajeto com pontos de interesse e estimativa de distância
- **Armazenado em:** tabela `route_suggestions`, associada ao `run_id`